

Du bon usage de `notify()` et `notifyAll()`

Fabrice.Kordon@lip6.fr



Quelles différences?

2

notify()

-  Associé au verrou d'un objet
-  Réveille **un seul** processus en attente sur la file associée au verrou

notifyAll()

-  Associé au verrou d'un objet
-  Réveille **tous les** processus en attente sur la file associée au verrou

Conseil d'usage

-  notifyAll()
-  Pourquoi?
 - ▶ notifyAll réveille tout le monde ⇒ ordonnancement équitable
 - ▶ notify() réveille «quelqu'un» ⇒ famine possible
-  Conséquences
 - ▶ Les appels à wait() doivent (généralement) être encapsulés dans une boucle

Quelles différences?

2

notify()

- Associé au verrou d'un objet
- Réveille **un seul** processus en attente sur la file associée au verrou

notifyAll()

- Associé au verrou d'un objet
- Réveille **tous** les processus en attente sur la file associée au verrou

Conseil

- notifyAll
- Pourquoi?



Recommandation

C'est Oracle qui le dit



- ▶ notifyAll réveille tout le monde ⇒ ordonnancement équitable
- ▶ notify() réveille «quelqu'un» ⇒ famine possible
- Conséquences
 - ▶ Les appels à wait() doivent (généralement) être encapsulés dans une boucle

Revoir LecteurEcrivain (classe MoniteurVariable)

3

LecteurEcrivain1

```
class MoniteurVariable {
    public int valeurEcrire;

    private boolean termine;
    private final Object fini; // doit-être stable ;- )

    MoniteurVariable() {
        fini = new Object();
        termine = false;
    }

    public void finEcrire() {
        synchronized (fini) {
            termine = true;
            fini.notify(); // doit être dans un bloc synchronized
        }
    }

    public void debutLire() throws InterruptedException {
        synchronized (fini) {
            if (! termine) {
                fini.wait(); // doit être dans un bloc synchronized
            }
        }
    }
}
```

Revoir LecteurEcrivain (classe MoniteurVariable)

3

LecteurEcrivain3

```
class MoniteurVariable {
    public int valeurEcrire;

    private boolean termine;
    private final Object fini; // doit-être stable ;- )

    MoniteurVariable() {
        fini = new Object();
        termine = false;
    }

    public void finEcrire() {
        synchronized (fini) {
            termine = true;
            fini.notifyAll(); // doit être dans un bloc synchronized
        }
    }

    public void debutLire() throws InterruptedException {
        synchronized (fini) {
            while (! termine) {
                fini.wait(); // doit être dans un bloc synchronized
            }
        }
    }
}
```

Revoir LecteurEcrivain (classe MoniteurVariable)

3

```
class MoniteurVariable {  
    public int valeurEcrive;  
  
    private boolean termine;  
    private final Object fini; // doit-être s  
  
    MoniteurVariable() {  
        fini = new Object();  
        termine = false;  
    }  
  
    public void finEcrire() {  
        synchronized (fini) {  
            termine = true;  
            fini.notifyAll(); // doit  
        }  
    }  
  
    public void finLire() {  
        synchronized (fini) {  
            while (!termine) {  
                try {  
                    fini.wait();  
                } catch (InterruptedException e) {}  
            }  
            valeurEcrive++;  
        }  
    }  
}
```

LecteurEcrivain3



Ne change rien ici
car 1 lecteur et 1 écrivain

**Fonctionnement
identique!**



Rappel

C'est Oracle qui le veut



synchronized

En guise de conclusion...

4

Les conseils de «Monsieur Oracle»

-  Toujours faire un `notifyAll()`
-  Toujours encapsuler les `wait()` dans une boucle

Il y a une raison...

-  Éviter la famine...
 - ▶ **Ordre du réveil non garanti...**

ORACLE®

